```
import random

# 1. Define the System Constraints & Variables
# A "Policy Package" chromosome consists of 3 genes: [Industrialization, Healt
# Each gene is a value from 0.0 to 1.0
POPULATION_SIZE = 20
GENERATIONS = 50
MUTATION_RATE = 0.1

def generate_initial_policy():
    return [random.random() for _ in range(3)]

# 2. The Fitness Function (The Bhutan vs. Standard Model)
def evaluate_fitness(policy):
    industrialization, healthcare, green_space = policy[0], policy[1], policy[

    # Calculate Economic Output
    economy = (industrialization * 0.7) + (healthcare * 0.3) - (green_space * (

    # Calculate Public Health Output (Clean air + healthcare vs industrial poll
    health = (healthcare * 0.6) + (green_space * 0.5) - (industrialization * 0

    # Bhutan's Fitness Function: Balances both, heavily penalizing extreme heal
    if health < 0.3:
        return 0.0  # System failure / Unacceptable strategy

    return (economy * 0.4) + (health * 0.6)

# 3. Selection: Choosing the healthiest policies to breed
def select_parent(population):
    # Tournament selection: pick two random policies, keep the one with higher
    p1, p2 = random.choice(population), random.choice(population)
    return p1 if evaluate_fitness(p1) > evaluate_fitness(p2) else p2

# 4. Crossover & Mutation: Creating the next generation of strategies
def crossover(parent1, parent2):
    # Swap traits between two successful systems
    return [parent1[0], parent2[1], parent1[2]]

def mutate(policy):
    # Apply small environmental policy shifts (mutations)
    for i in range(len(policy)):
        if random.random() < MUTATION_RATE:
            policy[i] = min(max(policy[i] + random.uniform(-0.1, 0.1), 0.0), 1
    return policy

# --- Main Simulation Loop ---
population = [generate_initial_policy() for _ in range(POPULATION_SIZE)]

for generation in range(GENERATIONS):
    # Score the entire population
    population = sorted(population, key=lambda x: evaluate_fitness(x), reverse=

    # Breed the next generation
    next_generation = [population[0], population[1]] # Keep top 2 automatically
    while len(next_generation) < POPULATION_SIZE:
        p1 = select_parent(population)
        p2 = select_parent(population)
```

```
            child = crossover(p1, p2)
            child = mutate(child)
            next_generation.append(child)

        population = next_generation

    # View the Optimized Result
    optimal_policy = max(population, key=evaluate_fitness)
    print(f"Optimized Strategy Found:\nIndustrialization: {optimal_policy[0]:.2f}\n
```

```
Optimized Strategy Found:
Industrialization: 1.00
Healthcare Funding: 1.00
Green Space: 1.00
```